

Resolução - Lista 3

Otimização de RNA

Considerando:

$$f(x_1, x_2) = x_1^4 + x_2^4 + x_1^2 x_2 + x_1 x_2^2 - 20x_1^2 - 15x_2^2$$

```
#install.packages("plotly")  
#install.packages("tidyverse")
```

```
library(plotly)
```

```
## Loading required package: ggplot2
```

```
##
```

```
## Attaching package: 'plotly'
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##      last_plot
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      filter
```

```
## The following object is masked from 'package:graphics':
```

```
##
```

```
##      layout
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.6
```

```
## v forcats    1.0.1      v stringr    1.6.0
```

```
## v lubridate  1.9.4      v tibble     3.3.0
```

```
## v purrr      1.2.0      v tidyr      1.3.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks plotly::filter(), stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
f <- function(x1, x2){
  x1^4 + x2^4 + x1^2*x2 + x1*x2^2 - 20*x1^2 - 15*x2^2
}
```

```
# grade (ajuste o intervalo se quiser)
x1 <- seq(-4, 4, length.out = 160)
x2 <- seq(-4, 4, length.out = 160)

# matriz z: linhas = x1, colunas = x2
z <- outer(x1, x2, f)

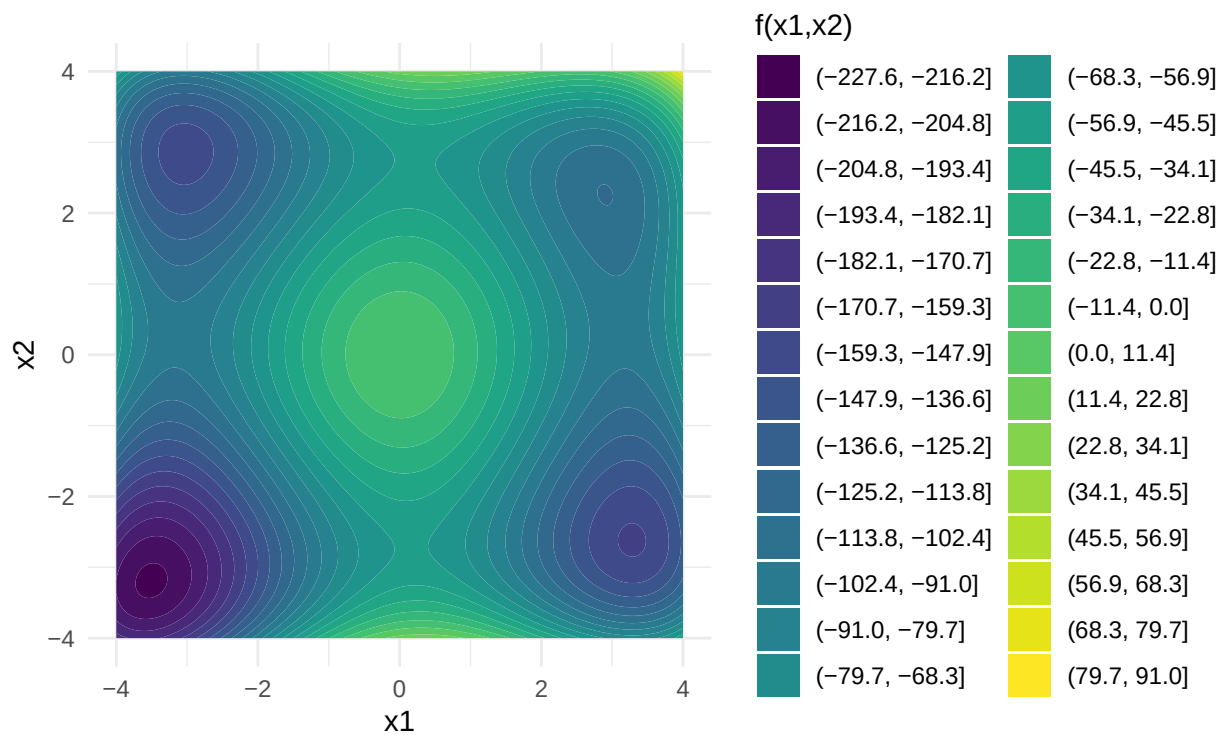
plot_ly(x = x1, y = x2, z = z, type = "surface") |>
  layout(
    scene = list(
      xaxis = list(title = "x1"),
      yaxis = list(title = "x2"),
      zaxis = list(title = "f(x1,x2)")
    )
  )
```

Letra A

Curvas de nível de $f(x_1, x_2)$

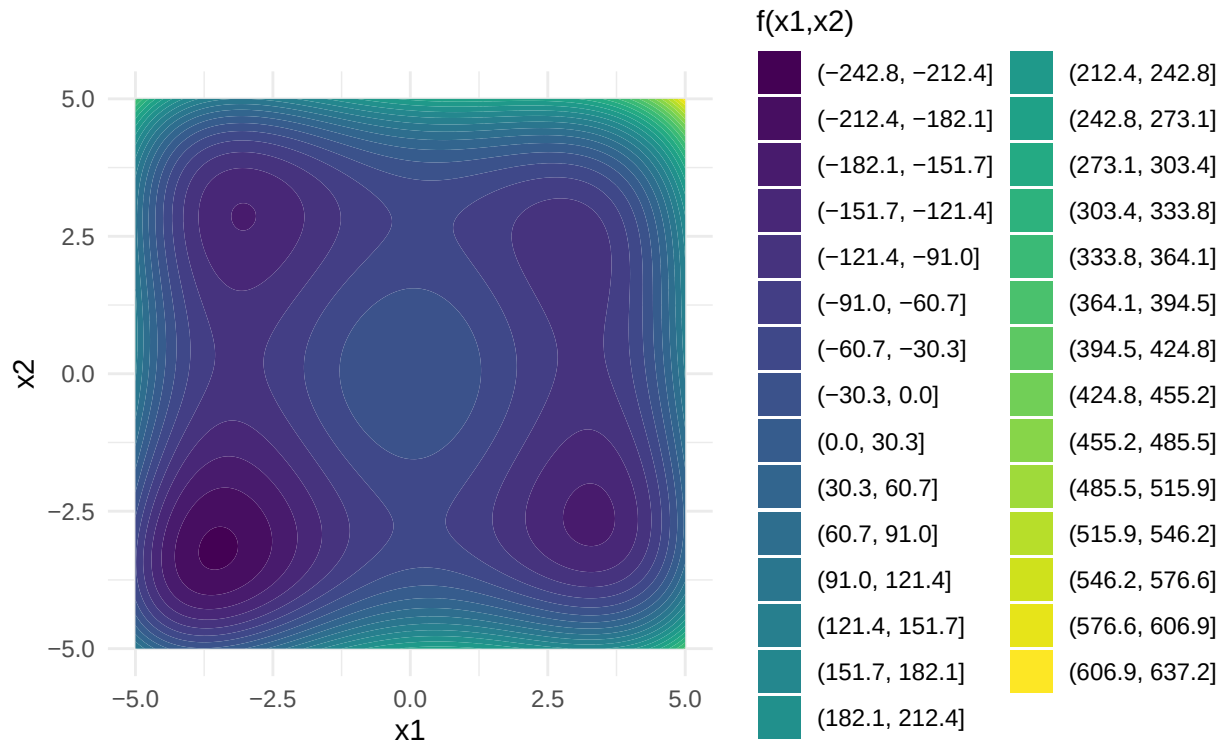
```
grid <- expand.grid(
  x1 = seq(-4, 4, by = 0.02),
  x2 = seq(-4, 4, by = 0.02)
) |>
  as_tibble() |>
  mutate(z = f(x1, x2))

ggplot(grid, aes(x = x1, y = x2, z = z)) +
  geom_contour_filled(bins = 30) +
  coord_equal() +
  labs(x = "x1", y = "x2", fill = "f(x1,x2)") +
  theme_minimal()
```

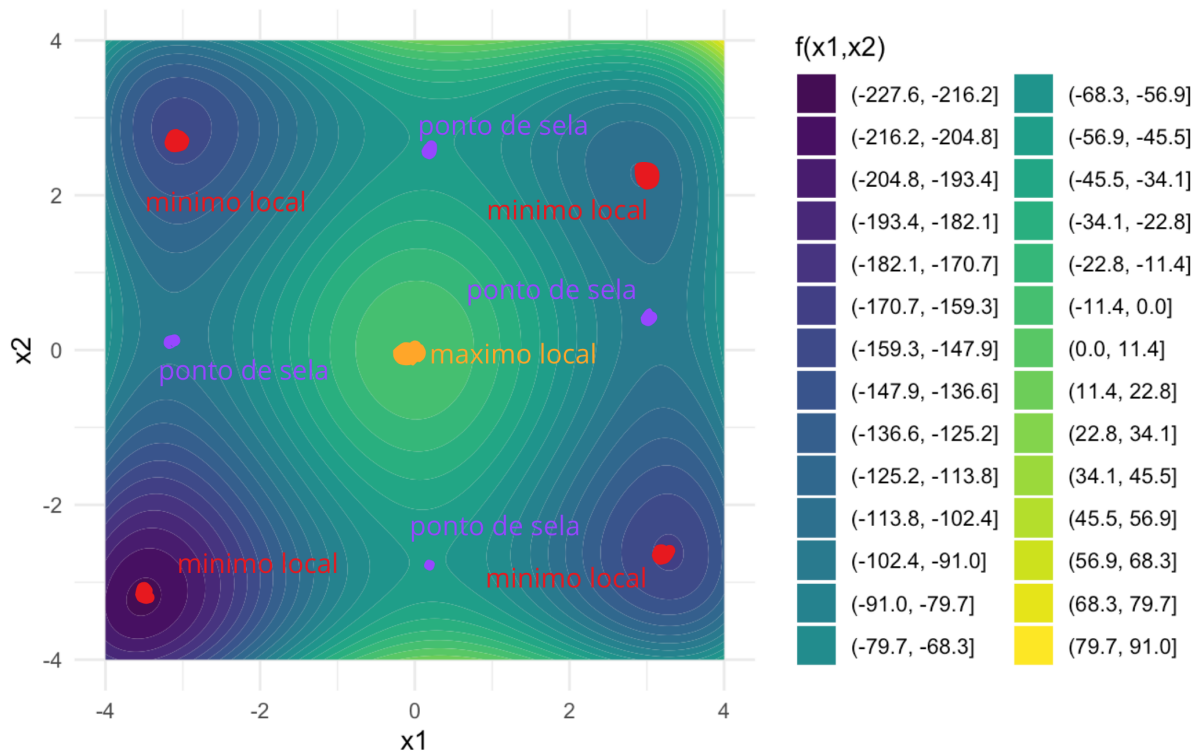


```
grid <- expand.grid(
  x1 = seq(-5, 5, by = 0.02),
  x2 = seq(-5, 5, by = 0.02)
) |>
  as_tibble() |>
  mutate(z = f(x1, x2))

ggplot(grid, aes(x = x1, y = x2, z = z)) +
  geom_contour_filled(bins = 30) +
  coord_equal() +
  labs(x = "x1", y = "x2", fill = "f(x1,x2)") +
  theme_minimal()
```



Rabiscando na mão os pontos de interesse



Letra B

$$\nabla_x f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right).$$

(Ver resolução no papel e caneta)

Letra C

Função computacional que implementa o método de gradiente para minimizar a função.

Tem que ter:

- taxa de aprendizado
- número de passos

- ponto de partida

Ver resolução prévia que eu fiz na mão para mais detalhes.

```
# derivada parcial 1
df_x1 <- function(x1, x2){
  return(4*x1^3 + 2*x1*x2 + x2^2 - 40*x1)
}

# derivada parcial 2
df_x2 <- function(x1, x2){
  return(4*x2^3 + x1^2 + 2*x1*x2 - 30*x2)
}

# em um k passo, taxa de aprendizado multiplicado por derivadas parciais naquele valor k
grad_step <- function(alpha, x1_k, x2_k){

  g1 <- df_x1(x1_k, x2_k)
  g2 <- df_x2(x1_k, x2_k)

  return(list(
    result_1 = alpha * g1,
    result_2 = alpha * g2
  ))
}
```

```
# testando
print(df_x1(0,5))
```

```
## [1] 25
```

```
print(df_x2(0,5))
```

```
## [1] 350
```

```
a <- grad_step(0.01,0,5)$result_1
print(a)
```

```
## [1] 0.25
```

```
b <- grad_step(0.01,0,5)$result_2
print(b)
```

```
## [1] 3.5
```

```
gradiente_min <- function(alpha, n_passos, x1_init, x2_init){

  hist <- matrix(NA, nrow = n_passos, ncol = 2)
  colnames(hist) <- c("x1", "x2")
}
```

```

# inicializo x1 e x2 com os iniciais
x1 <- x1_init
x2 <- x2_init

for (i in 1:n_passos) {

  # calculo alpha * derivada parcial 1 e alpha * derivada parcial 2
  step <- grad_step(alpha, x1, x2)

  # atualizo x1 e x2 com essa atualização de valor
  x1 <- x1 - step$result_1
  x2 <- x2 - step$result_2

  # histórico
  hist[i, ] <- c(x1, x2)

}

return(list(
  hist_min = hist,
  x1_min = x1,
  x2_min = x2
))
}

```

Letra D

```

x1_init = 0
x2_init = 5
alpha = 0.01
n_passos = 100

```

```
result <- gradiente_min(0.01, 100, 0, 5)
```

```
result$x1_min
```

```
## [1] -3.040141
```

```
result$x2_min
```

```
## [1] 2.865981
```

```

hist_df <- as_tibble(result$hist_min) |>
  mutate(iter = row_number())

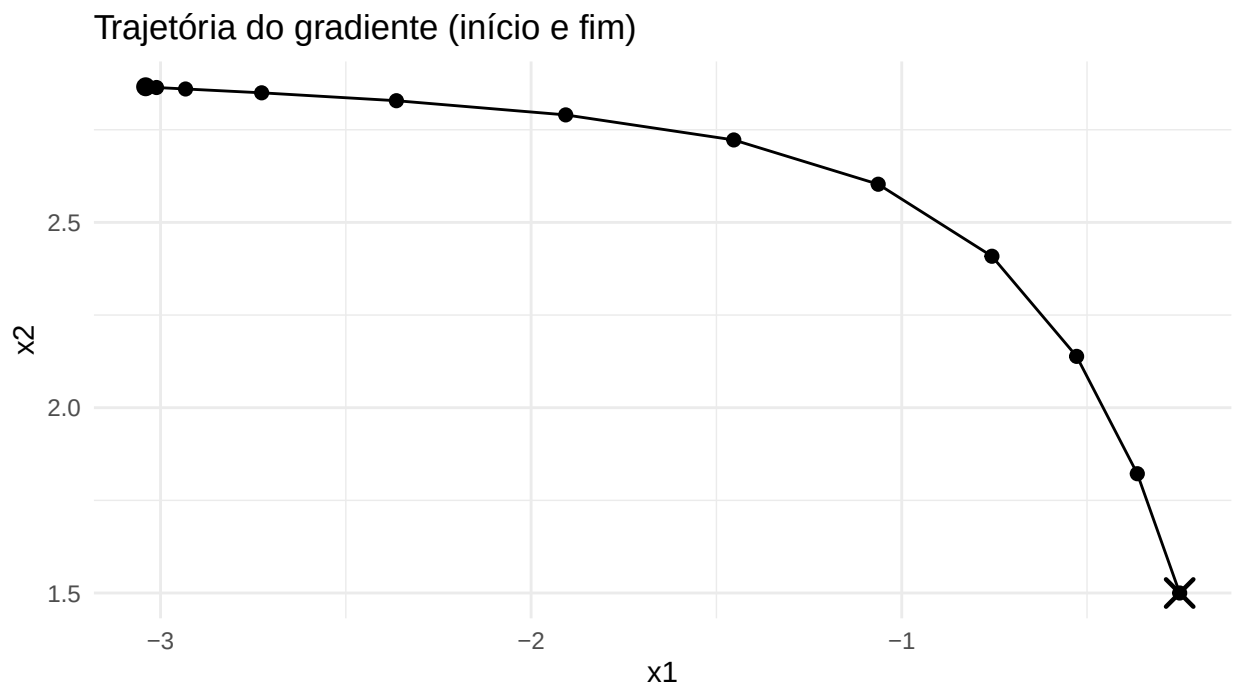
# trajetória (x1 vs x2)
ggplot(hist_df, aes(x = x1, y = x2)) +
  geom_path() +
  geom_point(size = 2) +

```

```

annotate(
  "point",
  x = hist_df$x1[1],
  y = hist_df$x2[1],
  shape = 4,
  size = 4,
  stroke = 1.2
) +
annotate(
  "point",
  x = hist_df$x1[nrow(hist_df)],
  y = hist_df$x2[nrow(hist_df)],
  shape = 16,
  size = 3
) +
coord_equal() +
labs(
  x = "x1",
  y = "x2",
  title = "Trajetória do gradiente (início e fim)"
) +
theme_minimal()

```



Letra E

Agora variando taxa de aprendizado:

- 1
- 0.1
- 0.01
- 0.001
- 0.0001

```
result1 <- gradiente_min(1, 100, 0, 5)
result2 <- gradiente_min(0.1, 100, 0, 5)
result3 <- gradiente_min(0.01, 100, 0, 5)
result4 <- gradiente_min(0.001, 100, 0, 5)
result5 <- gradiente_min(0.0001, 100, 0, 5)
```

```
hist_df1 <- as_tibble(result1$hist_min) |>
  mutate(iter = row_number())
```

```
hist_df2 <- as_tibble(result2$hist_min) |>
  mutate(iter = row_number())
```

```
hist_df3 <- as_tibble(result3$hist_min) |>
  mutate(iter = row_number())
```

```
hist_df4 <- as_tibble(result4$hist_min) |>
  mutate(iter = row_number())
```

```
hist_df5 <- as_tibble(result5$hist_min) |>
  mutate(iter = row_number())
```

```
# 1
```

```
hist_df1 <- hist_df1 |>
  filter(is.finite(x1), is.finite(x2))
```

```
ggplot(hist_df1, aes(x = x1, y = x2)) +
  geom_path() +
  geom_point(size = 2) +
  annotate(
    "point",
    x = hist_df1$x1[1],
    y = hist_df1$x2[1],
    shape = 4,
    size = 4,
    stroke = 1.2
  ) +
  annotate(
    "point",
```

```

x = hist_df1$x1[nrow(hist_df)],
y = hist_df1$x2[nrow(hist_df)],
shape = 16,
size = 3
) +
coord_equal() +
labs(
  x = "x1",
  y = "x2",
  title = "Trajetória do gradiente (1)"
) +
theme_minimal()

```

```

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

```



```

# 0.1

hist_df2 <- hist_df2 |>
  filter(is.finite(x1), is.finite(x2))

ggplot(hist_df2, aes(x = x1, y = x2)) +
  geom_path() +
  geom_point(size = 2) +

```

```

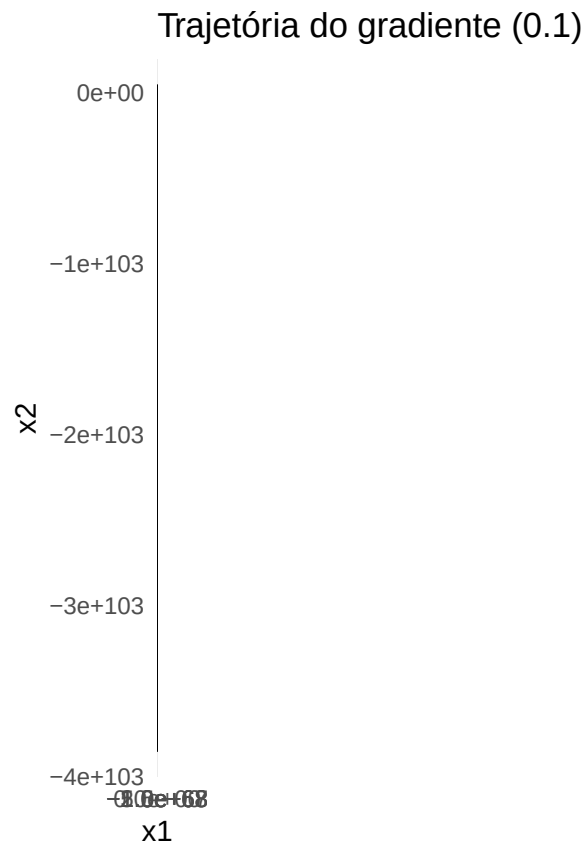
annotate(
  "point",
  x = hist_df2$x1[1],
  y = hist_df2$x2[1],
  shape = 4,
  size = 4,
  stroke = 1.2
) +
annotate(
  "point",
  x = hist_df2$x1[nrow(hist_df)],
  y = hist_df2$x2[nrow(hist_df)],
  shape = 16,
  size = 3
) +
coord_equal() +
labs(
  x = "x1",
  y = "x2",
  title = "Trajetória do gradiente (0.1)"
) +
theme_minimal()

```

```

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

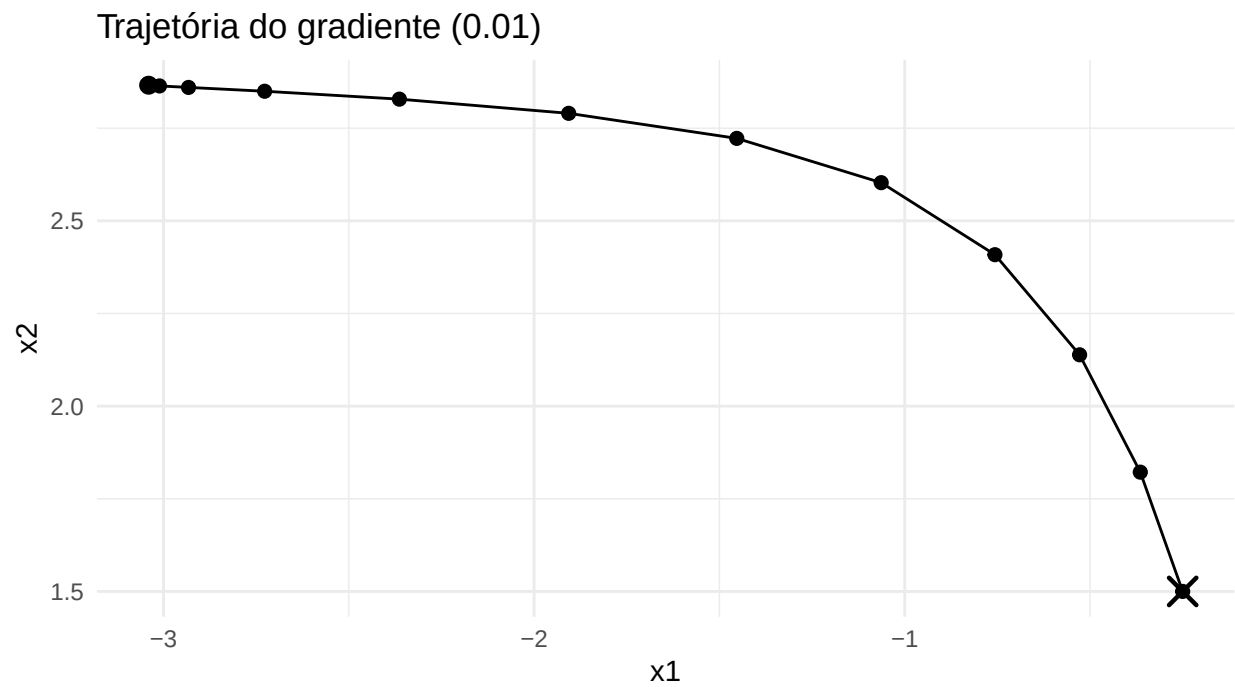
```



```

# 0.01
ggplot(hist_df3, aes(x = x1, y = x2)) +
  geom_path() +
  geom_point(size = 2) +
  annotate(
    "point",
    x = hist_df3$x1[1],
    y = hist_df3$x2[1],
    shape = 4,
    size = 4,
    stroke = 1.2
  ) +
  annotate(
    "point",
    x = hist_df3$x1[nrow(hist_df)],
    y = hist_df3$x2[nrow(hist_df)],
    shape = 16,
    size = 3
  ) +
  coord_equal() +
  labs(
    x = "x1",
    y = "x2",
    title = "Trajetória do gradiente (0.01)"
  ) +
  theme_minimal()

```



```
# 0.001

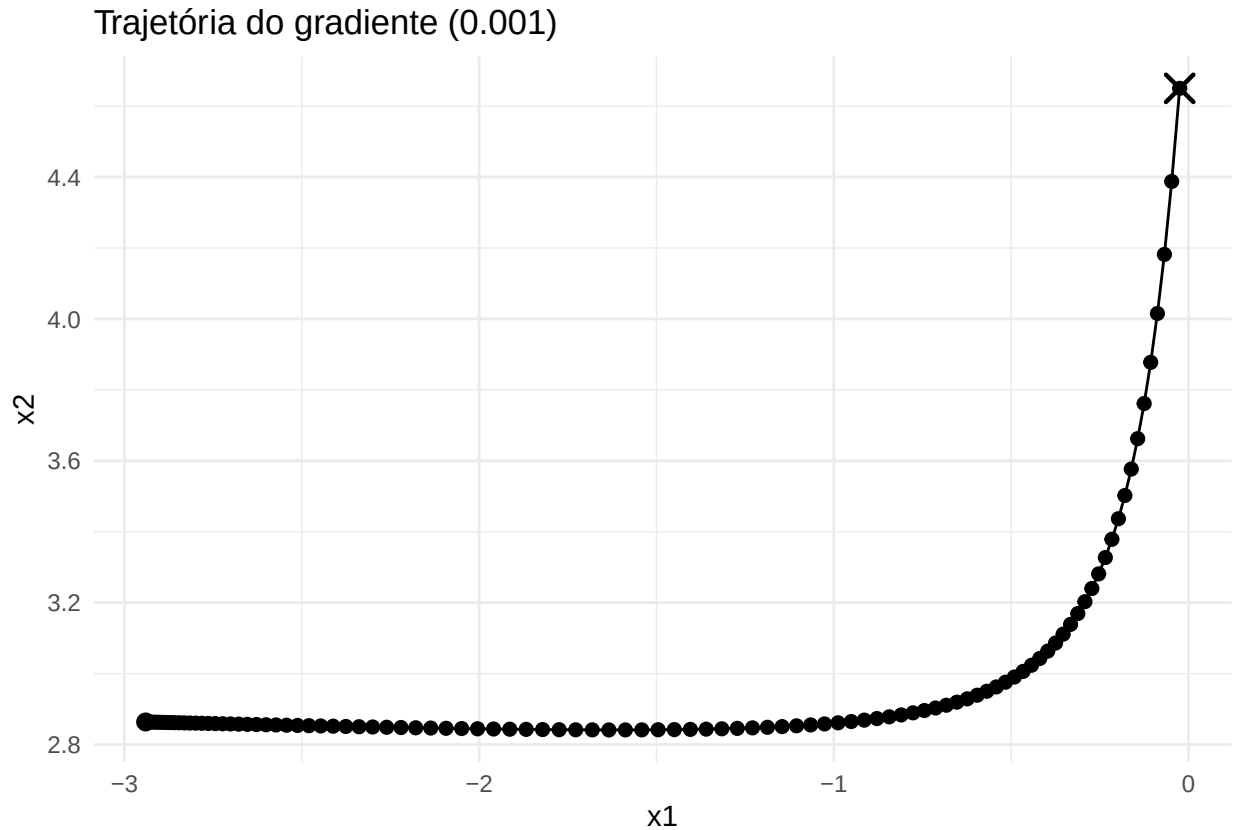
hist_df4 <- hist_df4 |>
  filter(is.finite(x1), is.finite(x2))

ggplot(hist_df4, aes(x = x1, y = x2)) +
  geom_path() +
  geom_point(size = 2) +
  annotate(
    "point",
    x = hist_df4$x1[1],
    y = hist_df4$x2[1],
    shape = 4,
    size = 4,
    stroke = 1.2
  ) +
  annotate(
    "point",
    x = hist_df4$x1[nrow(hist_df)],
    y = hist_df4$x2[nrow(hist_df)],
    shape = 16,
    size = 3
  ) +
  coord_equal() +
  labs(
    x = "x1",
```

```

y = "x2",
title = "Trajetória do gradiente (0.001)"
) +
theme_minimal()

```



```

# 0.0001

# limpeza mínima (evita NA/Inf)
hist_df5_ok <- hist_df5 |>
  filter(is.finite(x1), is.finite(x2))

n <- nrow(hist_df5_ok)

ggplot(hist_df5_ok, aes(x = x1, y = x2)) +
  geom_path(linewidth = 1) +
  geom_point(size = 1.2) +
  annotate(
    "point",
    x = hist_df5_ok$x1[1],
    y = hist_df5_ok$x2[1],
    shape = 4,
    size = 4,
    stroke = 1.2
  ) +
  annotate(

```

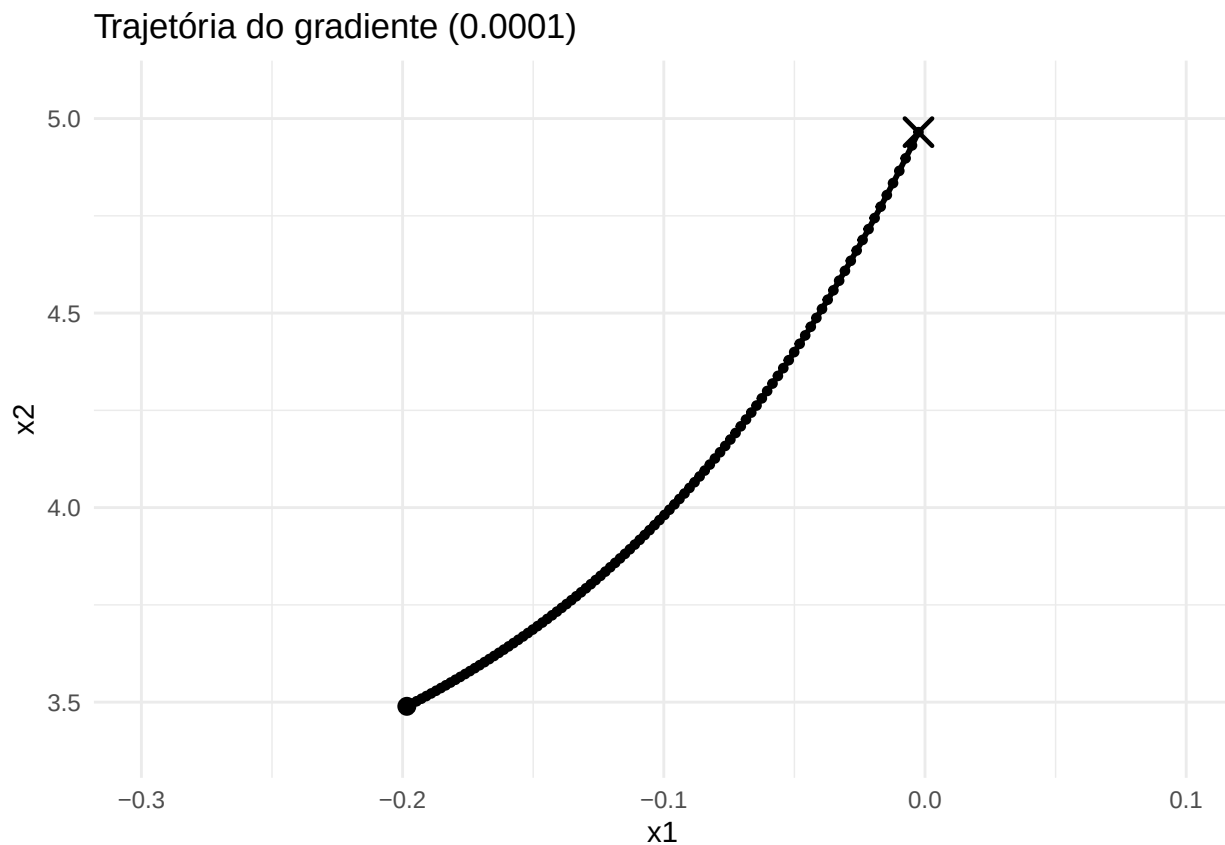
```

"point",
x = hist_df5_ok$x1[n],
y = hist_df5_ok$x2[n],
shape = 16,
size = 3
) +
coord_equal() +
coord_cartesian(
  xlim = range(hist_df5_ok$x1) + c(-0.1, 0.1),
  ylim = range(hist_df5_ok$x2) + c(-0.1, 0.1)
) +
labs(
  x = "x1",
  y = "x2",
  title = "Trajetória do gradiente (0.0001)"
) +
theme_minimal()

```

Coordinate system already present.

i Adding new coordinate system, which will replace the existing one.



Taxas de aprendizado

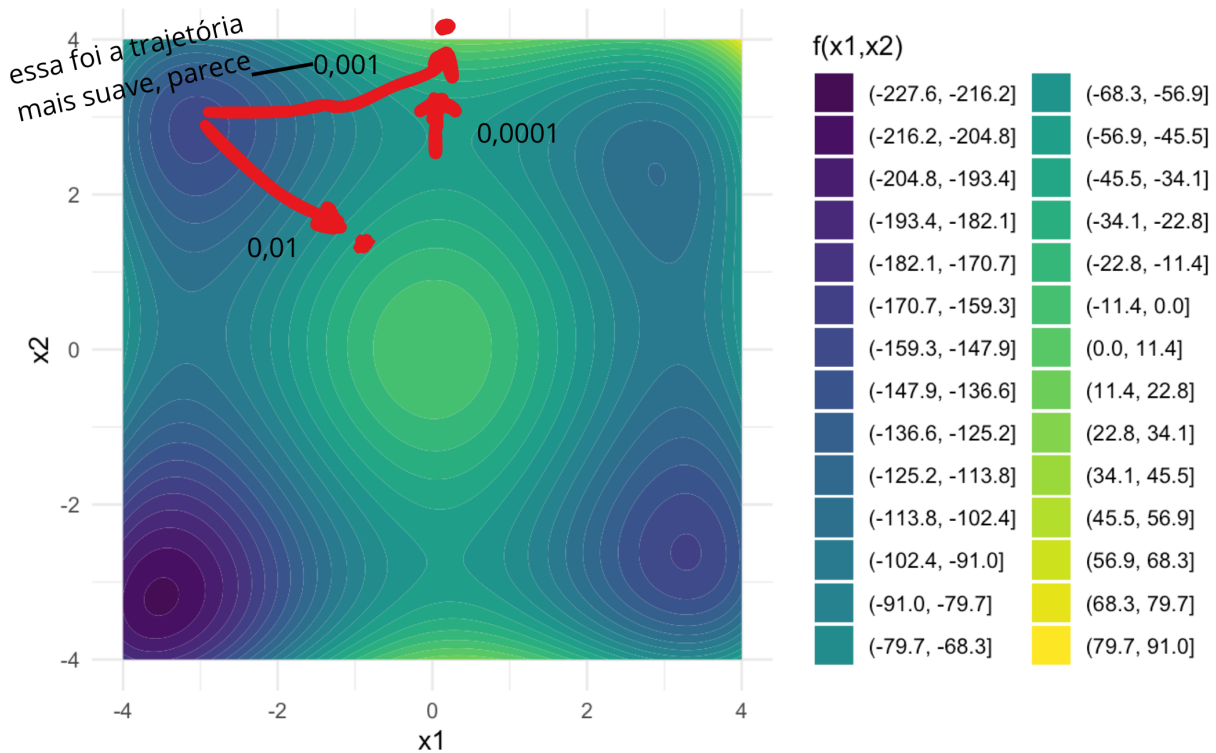
1 - divergiu

0.1 - divergiu

0.01 - passo muito grande?

0.001 - mais suave, segue curva do vale <- esse seria o valor escolhido!

0.0001 - mais lento, estável



Letra F

```
set.seed(123)

iniciais <- tibble(
  id = 1:20,
  x1_init = runif(20, min = -5, max = 5),
  x2_init = runif(20, min = -5, max = 5)
)
```

```
alpha <- 0.001
n_passos <- 5000

# objetos para guardar resultados finais
x1_final <- numeric(20)
x2_final <- numeric(20)

hist_all <- data.frame()

for (i in 1:20) {
  res <- gradiente_min(
```



```

    alpha = alpha,
    n_passos = n_passos,
    x1_init = iniciais$x1_init[i],
    x2_init = iniciais$x2_init[i]
  )

  x1_final[i] <- res$x1_min
  x2_final[i] <- res$x2_min

  hist_i <- as.data.frame(res$hist_min)
  colnames(hist_i) <- c("x1", "x2")

  hist_i$run <- i
  hist_i$iter <- 1:nrow(hist_i)

  hist_all <- rbind(hist_all, hist_i)
}

```

```

resultados <- data.frame(
  x1_init = iniciais$x1_init,
  x2_init = iniciais$x2_init,
  x1_final = x1_final,
  x2_final = x2_final
)

```

resultados

```

##      x1_init    x2_init  x1_final  x2_final
## 1 -2.1242248  3.8953932 -3.040141  2.865981
## 2  2.8830514  1.9280341  2.903159  2.262064
## 3 -0.9102308  1.4050681 -3.040141  2.865981
## 4  3.8301740  4.9426978  2.903159  2.262064
## 5  4.4046728  1.5570580  2.903159  2.262064
## 6 -4.5444350  2.0853047 -3.040141  2.865981
## 7  0.2810549  0.4406602  2.903159  2.262064
## 8  3.9241904  0.9414202  2.903159  2.262064
## 9  0.5143501 -2.1084026  3.284497 -2.624033
## 10 -0.4338526 -3.5288635 -3.510853 -3.196775
## 11  4.5683335  4.6302423  2.903159  2.262064
## 12 -0.4666584  4.0229905 -3.040141  2.865981
## 13  1.7757064  1.9070528  2.903159  2.262064
## 14  0.7263340  2.9546742  2.903159  2.262064
## 15 -3.9707532 -4.7538632 -3.510853 -3.196775
## 16  3.9982497 -0.2220403  3.284497 -2.624033
## 17 -2.5391227  2.5845954 -3.040141  2.865981
## 18 -4.5794047 -2.8359206 -3.510853 -3.196775
## 19 -1.7207928 -1.8181899 -3.510853 -3.196775
## 20  4.5450365 -2.6837421  3.284497 -2.624033

```

```

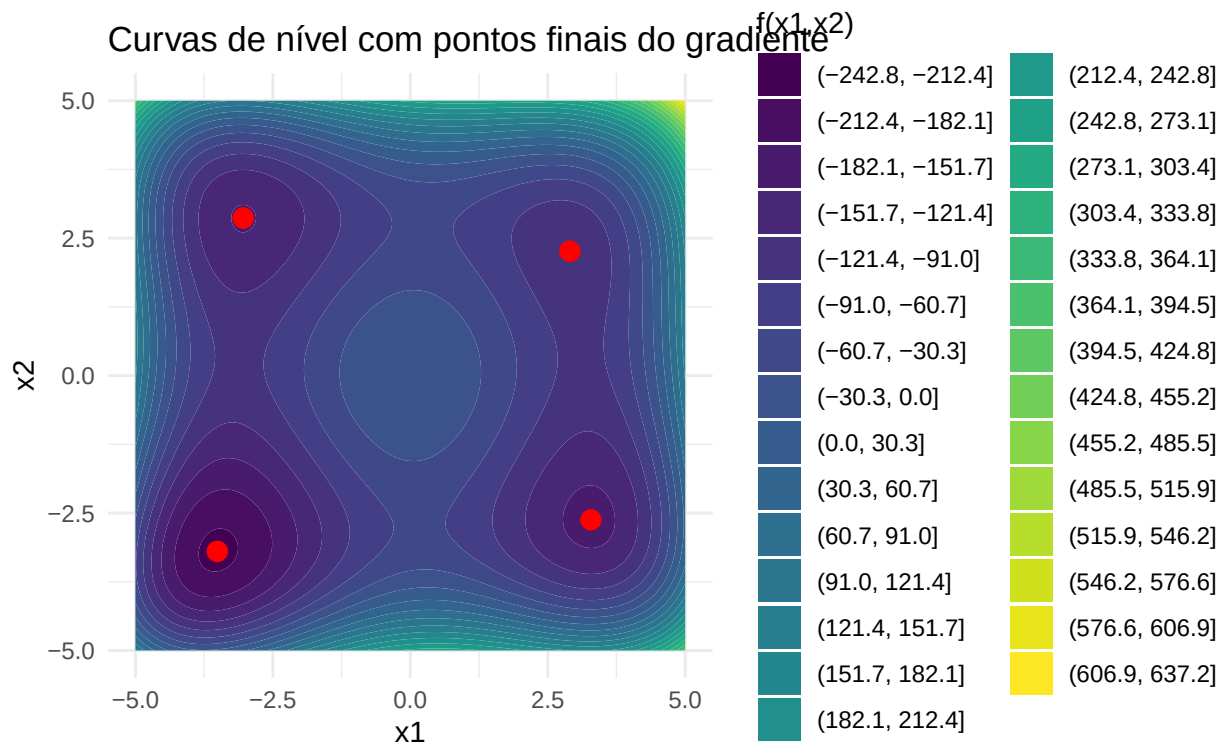
ggplot(grid, aes(x = x1, y = x2, z = z)) +
  geom_contour_filled(bins = 30) +
  geom_point(
    data = resultados,

```

```

aes(x = x1_final, y = x2_final),
inherit.aes = FALSE,
color = "red",
size = 3
) +
coord_equal() +
labs(
  x = "x1",
  y = "x2",
  fill = "f(x1,x2)",
  title = "Curvas de nível com pontos finais do gradiente"
) +
theme_minimal()

```



```

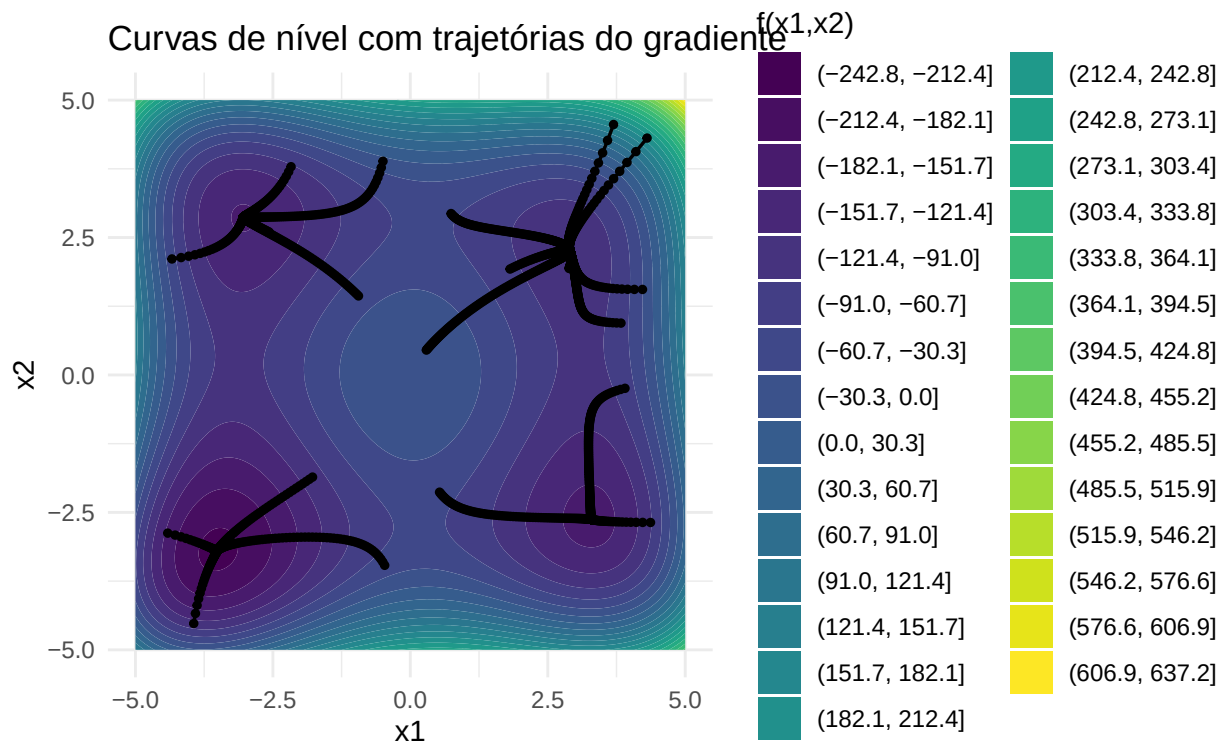
ggplot(grid, aes(x = x1, y = x2, z = z)) +
  geom_contour_filled(bins = 30) +
  geom_path(
    data = hist_all,
    aes(x = x1, y = x2, group = run),
    inherit.aes = FALSE,
    color = "black",
    linewidth = 0.6
  ) +
  geom_point(
    data = hist_all,

```

```

aes(x = x1, y = x2, group = run),
inherit.aes = FALSE,
color = "black",
size = 1
) +
coord_equal() +
labs(
  x = "x1",
  y = "x2",
  fill = "f(x1,x2)",
  title = "Curvas de nível com trajetórias do gradiente"
) +
theme_minimal()

```



Todos convergem, mas nem todos convergem para o mesmo mínimo!

Letra G

Repetindo letra D, fazendo gradiente com momento

Letra H

Repetindo letra D, fazendo gradiente RMSProp

Letra I

Repetindo letra D, fazendo gradiente Adam

Letra J

Grafico dos passos nas letras d, g, h, i